Phase-Adaptive Generation: Learned Compute Scheduling for Diffusion Language Models

Keshav Krishna Prajesh Kumar Siva Satvik Mandapati
New York University, Building LLM Reasoners — Spring 2026
{kk6081, pg2973, sm12779}@nyu.edu

Abstract

Diffusion language models decode by iteratively refining masked tokens, which exposes a useful scheduling question: should every part of a reasoning answer receive the same block size and refinement budget? We study this question through Phase-Adaptive Generation (PAG), a system for predicting block-level control tuples from generation traces and using them to adapt LLaDA decoding. The project produced three connected contributions. First, we built trace-analysis tooling for Dream and LLaDA, including token-level stabilization features, PELT/kernel change-point detection, and a Streamlit inspection interface. Second, we converted AdaBlock-style LLaDA traces into block-native training data and compared several next-control predictors, including a regression Transformer, a variable-order Markov baseline, a multi-output random forest, and a classification/ordinal Transformer over rich block features. Third, we integrated the learned predictor into a tuple-conditioned LLaDA scheduler and evaluated it against AdaBlock on 200 held-out GSM8K prompts. On the final run, PAG matched AdaBlock answer accuracy at 89.5% while reducing average total NFE from 90.800 to 72.070 per prompt, a $\approx 21.4\%$ reduction, with a small predictor overhead of 64.076 ms per prompt. PAG also gave significant response time savings over vanilla AdaBlock with average runtime delta of PAG - AdaBlock being approximately 0.10 sec.

1 Introduction

Autoregressive language models spend one decoding step per output token. Diffusion language models (dLLMs) such as Dream and LLaDA instead produce text through iterative masked-token refinement, so they can commit multiple positions in a block. This creates an explicit compute-allocation problem. Some spans of a solution are syntactically predictable or copied from context; other spans contain arithmetic, variable binding, or answer-finalization steps. A fixed schedule treats both regimes similarly.

PAG asks whether generation traces contain enough phase structure to forecast the next block-level control decision. Our final control tuple is

$$y_t = (\text{block\_size}_t,\ \text{refinement\_budget}_t),$$

where block size controls how many tokens are attempted together and refinement budget controls the number of model forward evaluations allowed before committing the block. The broader implementation supports richer conditioning fields: observed NFE, stabilization/refinement steps, confidence statistics, digit fraction, delimiter fraction, and gap statistics.

The project evolved in two phases. We first treated phase discovery as an interpretability problem over token-level traces: collect Dream/LLaDA refinement histories, extract stabilization features, and segment them with change-point detection (CPD). This showed interpretable difficulty shifts but also exposed a label-construction problem: CPD boundaries move with the feature, smoothing window, and penalty. We therefore shifted to a block-native path based on AdaBlock traces, where block boundaries and compute budgets are directly observed. CPD remains useful for analysis, while training and scheduling use block-level tuples.

2 Related Work

Diffusion language models. Dream and LLaDA demonstrate that diffusion-style language generation can support reasoning, planning, and instruction following without left-to-right token generation [Ye et al., 2025, Nie et al., 2025]. Block Diffusion formalizes semi-autoregressive diffusion decoding as an interpolation between fully autoregressive and fully parallel generation [Arriola et al., 2025]. DiffuLLaMA and related adaptation work show that diffusion LMs can inherit strong language-model priors from autoregressive models [Gong et al., 2025].

Efficient dLLM inference. Fast-dLLM studies KV caching and parallel decoding for training-free acceleration [Wu et al., 2026]. AdaBlock-dLLM is the closest prior system to our final implementation: it adapts block

length using confidence and semantic boundary heuristics [Lu et al., 2026]. Our contribution is complementary. Instead of only reacting to the current confidence band, we train predictors over realized block histories and feed their next-control predictions into the decoding loop.

Sequence modeling and change-point detection. Our predictor variants use compact Transformer encoders with sinusoidal positional encodings [Vaswani et al., 2017]. Our early analysis used PELT-style CPD, which optimizes a penalized segmentation objective and prunes dynamic-programming states for efficiency [Killick et al., 2012]. In this project, CPD is an analysis and data-understanding tool rather than the final source of scheduler labels.

3 System Overview

PAG comprises two subsystems: an offline predictor-training pipeline that extracts block-level control tuples from AdaBlock LLaDA generation traces and trains a Transformer-based next-tuple predictor, and an online scheduler that loads the trained checkpoint and governs per-block refinement with a learned budget and soft-cap early exit. Figure 1 shows this overview.

Predictor training. We run AdaBlock-dLLM with LLaDA-8B-Instruct on 5,000 GSM8K training problems, recording one control tuple per decoded block. Each tuple captures the realized block size, the number of model forward evaluations (NFE), the mean and minimum top-1 softmax confidence across unmasked positions, the fraction of digit tokens in the block, and the fraction of delimiter tokens. The resulting corpus contains 138,901 tuples. For training, we construct sliding-window sequences of length eight: each window of past tuples predicts the next (*block_size*, *refinement_budget*) pair. Features are z-score normalized per field using corpus statistics. The predictor is a Transformer encoder (ablations over layers, heads, learning rate, pre-norm) with two output heads: a softmax classifier over 128 block-size classes and an ordinal regression head over 83 binary refinement-budget thresholds. The budget head receives the block-size logits as conditioning, allowing the model to capture that larger blocks often need different budgets. Training uses cross-entropy for block size and binary cross-entropy for ordinal thresholds with an AdamW optimizer and early stopping. The final checkpoint bundles model weights, normalization statistics, and the ordered feature-field list.

Online scheduling. At decode time, the checkpoint is loaded by a scheduler that maintains a rolling context of realized tuples. The first block completely simulates AdaBlock to get block coundary and refinement; subsequent blocks pad the observed history to the window length, forward it through the predictor, and clamp the output by remaining sequence length and configured minimum and maximum bounds. Block sizing follows AdaBlock's adaptive mechanism: the model identifies the highest-confidence delimiter token in a forward-looking window and positions the block boundary there, defaulting to a system-chosen length when no delimiter meets the threshold. The predictor supplies the refinement budget independently. Decoding proceeds as a threshold-based unmasking loop within the active block. Once the forward-evaluation count reaches the predicted budget, a soft-cap mechanism checks per-token confidence and stability: if the minimum unmasking confidence exceeds $\tau_{\text{commit}} = 0.8$ or predictions have been stable for $\tau_{\text{stable}} = 2$ steps, the block exits early or when only a small fraction of the block remains and confidence is adequate, a forced commit terminates refinement unconditionally. After each block, the realized tuple—actual NFE, mean and minimum confidence, digit fraction, and delimiter fraction—is appended to the rolling context. The scheduler is a drop-in replacement within AdaBlock-dLLM's LLaDA generation harness, enabling per-prompt comparison against an unmodified AdaBlock run from the same random seed.

4 Trace Data and Interpretability

4.1 Token-Level CPD Analysis

Our first research thread instrumented Dream and LLaDA generation to store per-token refinement histories. At each diffusion step we record token identities, top-1 and top-2 softmax probabilities, and full-vocabulary entropy for every position. A per-token observation history is built by tracking positions across steps via their token index. From this history we identify each token's *stabilization step*: the earliest step index after which a token's identity never changes. All scalar features described below are read at this stabilization step, yielding one value per final output token.

Change-point detection (CPD) partitions a univariate series into contiguous segments whose internal statistical

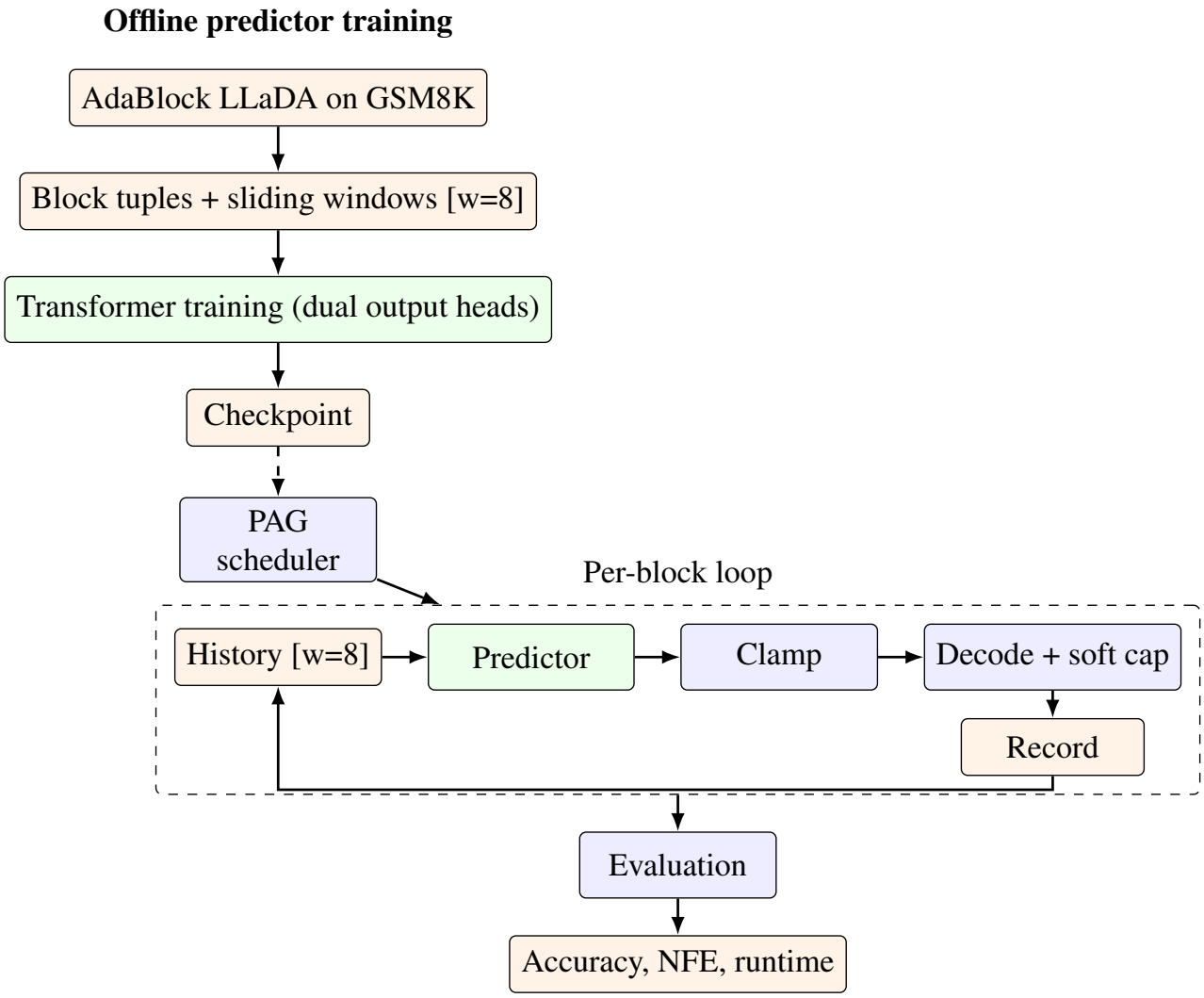


Figure 1: System architecture. The offline training pipeline (top) extracts block-level tuples from AdaBlock LLaDA traces and trains a Transformer predictor with dual output heads. The online scheduler (bottom) loads the checkpoint and governs per-block decoding via a learned refinement budget and soft-cap early exit.

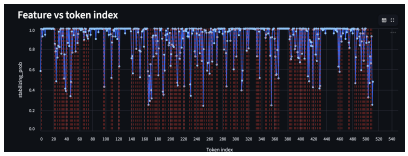
properties are homogeneous. Given a feature series $y_1, \ldots, y_n$ indexed by token position, the PELT algorithm (Pruned Exact Linear Time) solves

$$\min_{K,\tau_1,\ldots,\tau_K} \sum_{k=1}^{K+1} \mathcal{C}(y_{\tau_{k-1}+1:\tau_k}) + \beta K,$$

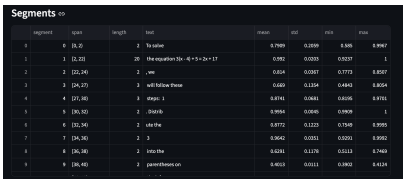
where $0 = \tau_0 < \tau_1 < \cdots < \tau_K < \tau_{K+1} = n$ are segment boundaries, $\beta \geq 0$ is a penalty that controls the trade-off between data fidelity and over-segmentation, and $\mathcal{C}$ is a segment cost function. With the ℓ_2 cost,

$$\mathcal{C}(y_{a:b}) = \sum_{t=a}^{b} (y_t - \hat{\mu})^2, \qquad \hat{\mu} = \frac{1}{b-a+1} \sum_{t=a}^{b} y_t,$$

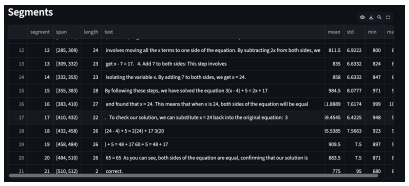
the total objective equals the sum of within-segment squared deviations plus a linear penalty on the number of changepoints. PELT achieves $O(n)$ runtime by pruning candidate boundaries that cannot improve the objective under the additive cost structure.



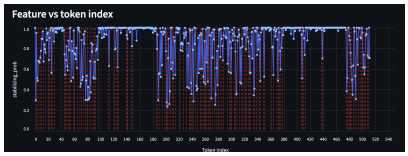
(a) Algebra: raw stabilizing probability.



(b) Algebra: probability with PELT segments.

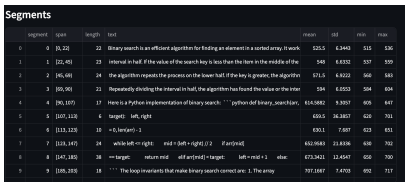


(c) Algebra: refinement-step signal.



(d) Binary search: raw stabilizing probability.

(e) Binary search: probability with PELT segments.



(f) Binary search: refinement-step signal.

Figure 2: Token-level CPD analysis for two representative prompts (algebra and binary search). Left column: raw stabilizing probability curves showing high-confidence plateaus alternating with low-confidence computation spans. Center column: same curves overlaid with PELT-detected segment boundaries. Right column: stabilizing refinement step, which is more monotonic across token position.

We extracted four per-token features at each token's stabilization step. **Stabilizing probability** ($p_{\text{top-1}}$) is the softmax confidence for the final token identity at the moment it stabilizes. **Stabilizing entropy** ($-\sum_i p_i \log p_i$) measures full-vocabulary uncertainty and is useful when top-1 probabilities saturate near 1.0. **Stabilizing margin** ($p_{\text{top-1}} - p_{\text{top-2}}$) captures the separation between the top two candidates. **Stabilizing refinement step** records the diffusion step index at which stabilization occurs, reflecting how long the model deliberated before committing.

Figure 2 shows these features for two representative Dream traces: an algebra problem and a binary-search code snippet. The left column plots raw stabilizing probability across token position. Both prompts exhibit a distinctive structure—extended plateaus near 1.0 for template tokens, repeated phrasing, and punctuation, punctuated by sharp dips to 0.3–0.6 during arithmetic computation, variable binding, and return-value construction. This aligns with the phase hypothesis: some spans are nearly effortless for the model while others demand significantly more selective refinement. The center column overlays PELT-detected boundaries on the same probability curves; segment boundaries align with intuitive semantic transitions such as the start of a numerical simplification or the boundary between a loop header and its body. The right column shows stabilizing refinement step, which is more monotonic: later tokens generally stabilize later in the diffusion process, making it less useful as a standalone segmentation signal.

A critical methodological observation tempered this analysis. CPD boundaries shift with the choice of feature, smoothing window, penalty value, and detector type—the same trace can produce anywhere from two to eight segments depending on configuration. This sensitivity makes CPD a valuable interpretability tool but an unreliable label generator for a scheduler whose control space is block-native. It motivated the shift to directly observed block-level traces, described next.

4.2 Block-Native AdaBlock Traces

The final training path uses AdaBlock-style LLaDA traces. We ran AdaBlock-dLLM with LLaDA-8B-Instruct on 5,000 GSM8K training problems. AdaBlock selects each block boundary adaptively: it identifies the highest-confidence delimiter token in a forward-looking window and positions the boundary there, defaulting to a fixed cap of 16 tokens when no delimiter exceeds the threshold. Within each block, LLaDA unmasking proceeds via threshold-based confidence sampling until the block's tokens stabilize. After every block we record a 12-field control tuple: the applied block size, the actual number of forward evaluations (NFE), per-token stabilization and refinement step statistics, mean and minimum top-1 softmax confidence, the fraction of digit tokens, and the fraction

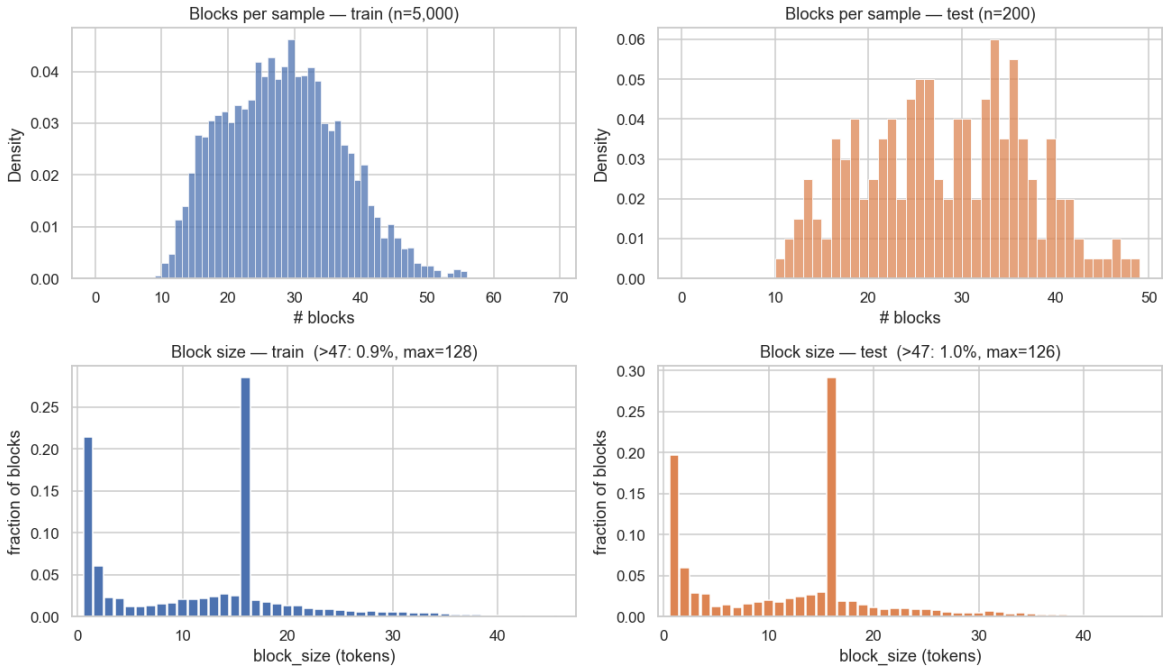


Figure 3: AdaBlock block structure on GSM8K. Block sizes have strong point masses at size 1 (delimiter-only blocks) and size 16 (AdaBlock's default content cap), with a long tail to larger sizes. Blocks per sample are roughly Gaussian centered at 22.

of delimiter tokens. The resulting corpus contains 138,901 tuples across training prompts and 5,543 tuples across 200 validation prompts. A separate 200-prompt test slice (GSM8K test indices 200–399) is held out for downstream scheduler evaluation.

Table 1: First 12 block tuples from a single GSM8K prompt. Delimiter blocks (size 1, NFE 1, confidence 1) alternate with content blocks. Blocks with nonzero digit fraction consume more NFE and exhibit lower minimum confidence.

Index	Block size	NFE	Min. $p_{\text{top-1}}$	Digit frac.	Delim frac.
0	22	4	0.903	0.000	0.045
1	1	1	1.000	0.000	1.000
2	16	9	0.675	0.062	0.000
3	1	1	1.000	0.000	1.000
4	16	9	0.418	0.062	0.000
5	16	11	0.618	0.062	0.062
6	16	9	0.445	0.062	0.062
7	8	5	0.897	0.000	0.250
8	10	5	0.581	0.000	0.100
9	1	1	1.000	0.000	1.000
10	20	2	0.845	0.050	0.050
11	13	3	0.803	0.154	0.154

Table 1 shows a contiguous 12-block slice from one prompt. The pattern is immediately visible: a size-1 block with confidence near 1.0 and NFE 1 marks each delimiter, while content blocks range from size 8 to 22 with NFE varying from 2 to 11. Blocks with nonzero digit fraction—those containing arithmetic—systematically require more forward evaluations and exhibit lower minimum confidence, sometimes falling below 0.5. This is the signal the predictor must learn: the same block size can demand very different compute depending on the token types inside it.

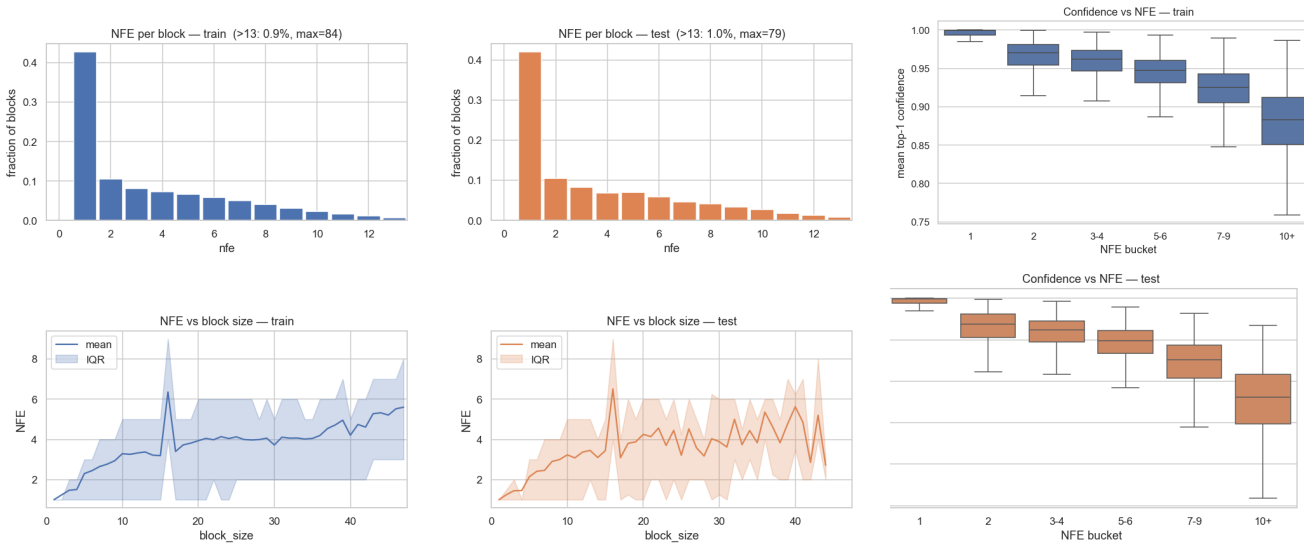


Figure 4: Per-block compute and confidence. NFE correlates positively with block size but exhibits enormous conditional variance—blocks of size 16 consume 3–15 forward passes. High-NFE blocks tend toward lower minimum confidence, confirming that the model's uncertainty drives the need for additional refinement.

Figure 3 confirms that the block-size distribution is fundamentally discrete and multimodal. The point mass at size 1 corresponds to isolated delimiters—newlines and commas that AdaBlock commits independently. The mass at size 16 is AdaBlock's default cap, applied when no delimiter meets the confidence threshold within the window. A tail extends to larger sizes (> 16) when consecutive low-confidence delimiters cause AdaBlock to merge spans. A regression model minimizing MSE would predict the conditional mean of these modes—for example, 8.5 for a prompt with equal numbers of size-1 and size-16 blocks—which corresponds to no feasible decoding outcome. This averaging failure, which we examine in §5.1, motivates the discrete classification and ordinal heads used in the final predictor.

Figure 4 examines the per-block compute-accuracy trade-off. While NFE correlates positively with block size, the conditional variance is enormous—blocks of size 16 consume anywhere from 3 to 15 forward passes. High-NFE blocks cluster at low minimum confidence, confirming that the model's uncertainty drives the need for additional refinement.

5 Predictors

5.1 Regression Transformer

The first learned predictor treated the next control tuple as a continuous regression target. A compact Transformer encoder consumed a sliding window of previous tuples, projected them to a hidden dimension, added sinusoidal position encodings, pooled the final context position, and produced real-valued outputs that were rounded to integers at inference time. Validation loss reached 0.096 on normalized targets, suggesting the model had learned the training distribution well.

This offline metric was misleading. When deployed inside the LLaDA decoding loop, the regression model produced block-size predictions that fell between the discrete modes of the empirical distribution—for example, 8 for a prompt with equal numbers of size-1 delimiter blocks and size-16 content blocks. These interpolated sizes have no training support: no real decoding block in the training corpus was created with those boundaries. LLaDA partially unmasked the requested span and left the remainder in the masked state, producing garbled output with truncated, repeated, or overlapping token spans. The resulting text exhibited what we term *degenerate English*: short windows of local fluency—a few correctly decoded words—embedded within broken sentences that carried no coherent meaning. The best checkpoint we obtained under this approach reached only 73% answer accuracy

on 30 held-out multi-step arithmetic prompts—compared to 93% for the unmodified AdaBlock baseline—while averaging just 8.1 blocks per prompt against AdaBlock's 12.3. The reduced block count indicates that the model merged semantically distinct spans into single oversized blocks. A representative output from the evaluation log shows clear degeneration: "' Number of bags: 3 bags bags - Number Number apples "' The text preserves isolated correct phrases but degrades into word repetitions, losing all coherent sentence structure.

Two root causes explain this collapse. First, MSE's Bayes-optimal prediction for a multimodal target is the conditional mean of the modes, which falls in a region of the control space with zero training density. The regression head has no inductive bias toward valid block sizes; it is equally happy to predict size 8 or size 16 if the squared error is similar. Second, the model was trained on rich tuples that include realized confidence and token-type fractions from past blocks. During deployment the *current* block's confidence is unknown until decoding completes, creating a distribution shift between the features seen at training time and those available at inference time.

This failure directly motivated the design described next. A classification head over block-size classes sidesteps this problem by construction: the softmax layer assigns a probability to each valid integer size (1 through 128), and argmax selects exactly one. There is no output unit for fractional sizes, so every prediction corresponds to a real decoding boundary. An ordinal head over refinement-budget thresholds respects the monotonic structure of NFE—more steps are always permissible, but the question is how many. After switching to this architecture, the same evaluation setting reached 87% answer accuracy with 23.4% total NFE savings—results that the regression model could not approach.

5.2 Classification/Ordinal Transformer

The final predictor keeps the Transformer encoder but replaces the output heads. The block-size head is a classifier over the set of possible block sizes. The refinement-budget head is ordinal: it predicts logits over binary thresholds and recovers the integer budget by counting thresholds whose sigmoid exceeds 0.5. The budget head is conditioned on block-size logits via concatenation, so the model can represent that larger blocks often need different refinement budgets. The complete training pipeline is shown in Figure 5.

Training optimizes a composite objective. Block-size prediction uses standard cross-entropy over $C = 128$ classes:

$$\mathcal{L}_{\text{block}} = -\sum_{c=1}^{C} \mathbf{1}_{(y_b=c)} \log \frac{\exp(z_{b,c})}{\sum_j \exp(z_{b,j})}$$

where z_b are the block-size logits. Refinement-budget prediction uses binary cross-entropy with logits over $T = 83$ ordinal thresholds:

$$\mathcal{L}_{\text{budget}} = -\frac{1}{T}\sum_{k=1}^{T} \left[w_k \cdot y_{r,k} \log \sigma(z_{r,k}) + (1 - y_{r,k}) \log(1 - \sigma(z_{r,k})) \right]$$

where σ is the logistic sigmoid and w_k are per-threshold positive weights computed from inverse class frequency, clamped to $[1, 10]$. At inference the budget is recovered as the number of thresholds whose sigmoid exceeds 0.5. The total loss is $\mathcal{L} = \mathcal{L}_{\text{block}} + \mathcal{L}_{\text{budget}}$.

The implementation supports a 6-feature and 12-feature input mode. The final scheduler checkpoint used the rich-feature classifier path and was integrated into the LLaDA decoding loop through the PAG tuple scheduler described in §6.

5.3 Other Predictor Experiments

We also explored two non-Transformer baselines.

Variable-order Markov model. The VOMM predictor memorizes recent tuple contexts and backs off when a context is unseen. It is useful as a sanity check because it is cheap and interpretable, but it cannot generalize well to continuous confidence features or unseen block patterns. We used it on early trace collections, and its accuracy was comparable to predicting the empirical mode from the training set—confirming that deterministic sequence matching alone was insufficient for this task.

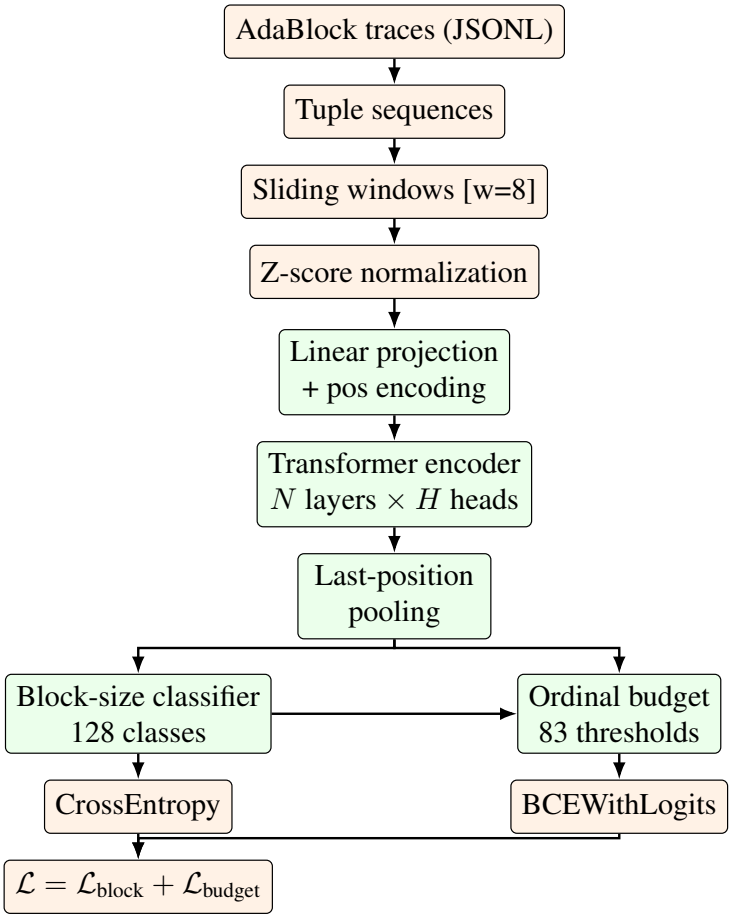


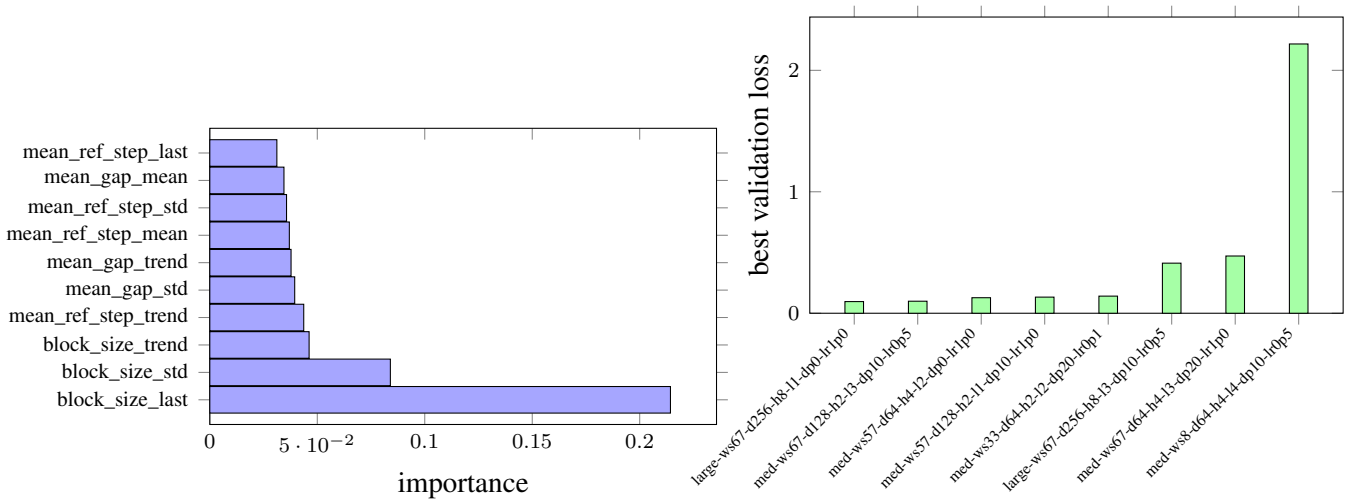
Figure 5: Comprehensive predictor training architecture. Data preparation (top) converts raw AdaBlock traces into normalized sliding-window sequences. The Transformer encoder (center) uses N layers, H attention heads, hidden dimension $d_{\text{model}} = 64$, and feed-forward dimension 256 with pre-norm. Hyperparameters N, H, context window size, dropout, and learning rate were ablated (see Fig. 6). The block-size head (bottom-left) is a 128-way classifier; the ordinal budget head (bottom-right) predicts 83 thresholds conditioned on block logits.

Random forest. A multi-output random forest (window size 5, 200 trees, max depth 15) trained on rolling-window statistics of past tuple features: { NFE, block size, max stabilization step, mean refinement step, mean gap, max gap }. With 90,994 training examples and 22,907 test examples, it achieved held-out MAE of 7.24 for block size and 1.66 for max stabilization step. Figure 6 shows that the most predictive feature was the last observed block size, followed by window variability and trend features. This importance ranking directly informed the design of the Transformer's input feature set and context window length, but the RF's lack of sequential structure made it a poor fit for the deployment loop.

6 Tuple-Conditioned Scheduling

The final scheduler integrates the predictor into LLaDA generation by combining AdaBlock's delimiter-based block boundaries with the predictor's refinement budget and a soft-cap early-exit mechanism. This design was the result of an iterative process that exposed several failure modes, described below.

Our first integration let the predictor decide both block boundaries and refinement budgets. The model frequently predicted $(\text{block\_size} = 1,\ \text{refinement} = 1)$—a single token with one forward pass. This triggered a *death spiral*: each tiny block produces an impoverished context tuple (nfe 1, confidence 1.0, no digits or delimiters), which skews the next prediction toward even smaller values, and the model's input distribution drifts further from the training distribution with every block. The output fragmented into long sequences of size-1 blocks with trivial refinement,



(a) Random forest feature importance.

(b) Best saved Transformer ablation checkpoints.

Figure 6: Predictor experiments beyond the final scheduler. The RF branch provided interpretable feature rankings; Transformer ablations tested context length, hidden size, heads, layers, dropout, and learning rate.

losing all coherent structure.

We introduced two countermeasures: a minimum block length of 4 and a recorded-NFE floor of 3 for content blocks. These prevented the worst fragmentation—the model could no longer produce single-token blocks—but they did not address the root cause. The clamped values had no analogue in the training distribution, so the predictor operated entirely out of distribution and became essentially useless. The hard refinement cap also meant that blocks containing genuinely difficult tokens were force-committed before converging, producing garbled predictions and explaining why the evaluation logs showed many blocks with stabilizing step 0—the token never had a chance to reach its final identity before the budget expired.

The core insight was that AdaBlock's delimiter-based block boundary detection solves a fundamentally different problem from what the predictor should forecast. AdaBlock identifies semantic boundaries by scanning softmax confidence at candidate delimiter positions—a reliable reactive signal based on the model's current logits. The predictor, by contrast, excels at forecasting the *computational effort* a block will require based on past history. We therefore separated the two concerns. Block sizes now come from AdaBlock's delimiter-based block-length routine, which scans a lookahead window and positions the boundary at the highest-confidence delimiter whose softmax probability exceeds a threshold. The predictor supplies only the refinement budget. Three soft exit gates make the budget adaptive without compromising the predictor's authority: they can only trigger *after* the predicted budget is reached. A **confidence gate** exits when every remaining masked token has confidence at least $\tau_{\text{commit}} = 0.8$. A **stability gate** exits when the set of predicted tokens at masked positions has been unchanged for $\tau_{\text{stable}} = 2$ consecutive steps. A **few-remaining gate** exits when less than 10% of the block remains and all remaining positions have adequate confidence. A hard cap at max_refinement_steps always applies as a safety ceiling.

Algorithm 1 Per-block PAG decoding with AdaBlock boundaries and soft-cap refinement.

procedure PAGDECODE($prompt,\ model,\ predictor$)
  $H \leftarrow [\,]$; $x \leftarrow$ MASKINIT($prompt$); $pos \leftarrow |prompt|$
  while $pos < gen\_length$ **do** ▷ Block boundary via AdaBlock delimiter scan
    $logits,\ tokens \leftarrow model(x)$
    $b \leftarrow$ CLAMP(ADABLOCKBOUNDARY($logits,\ tokens$))
    ▷ Predict refinement budget from history
    if $H = \varnothing$ **then**
      $r \leftarrow r_{\text{seed}}$
    else
      $r \leftarrow predictor(\text{PAD}(H)) + offset$
    end if
    $r \leftarrow$ CLAMP($r,\ min\_ref,\ max\_ref$)
    ▷ Block refinement with soft-cap gates
    $nfe \leftarrow 0$; $prev \leftarrow [\,]$
    while $\exists$ *masked token in* $[pos,\ pos + b)$ **do**
      $logits,\ tokens \leftarrow model(x)$
      $nfe \leftarrow nfe + 1$; UNMASKLOWCONF($x,\ tokens$)
      if $nfe \geq r$ **then**
        if $\min\_conf \geq 0.8$ **then**
          break ▷ confidence
        else if STABLE($prev,\ 2$) **then**
          break ▷ stability
        else if $frac < 0.1$ **then**
          break ▷ few remaining
        end if
      end if
      if $nfe \geq max\_ref$ **then**
        break ▷ hard cap
      end if
      $prev$.APPEND($tokens$)
    end while
    ▷ Record realized tuple and advance
    H.APPEND($(b,\ nfe,\ conf,\ digit,\ delim)$)
    $pos \leftarrow pos + b$
  end while
  return $x[pos\_start:]$
end procedure

7 Evaluation

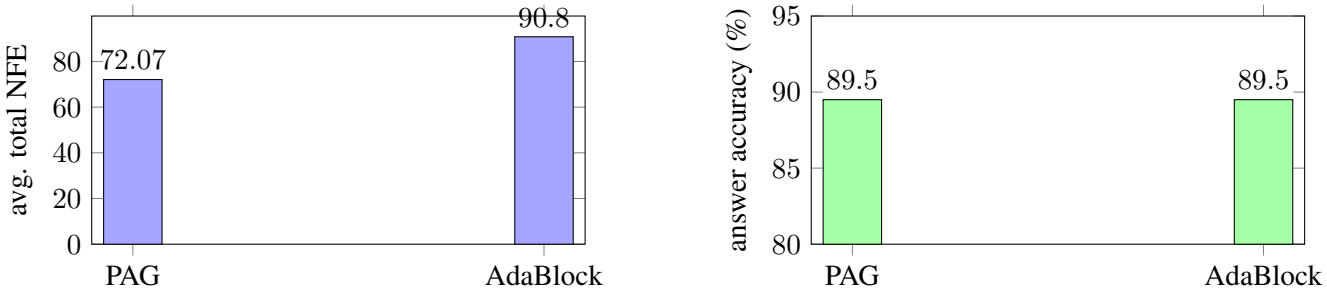
7.1 Setup

We evaluate on 200 GSM8K test prompts, `gsm8k_test_0200–gsm8k_test_0399`, using LLaDA-8B-Instruct with generation length 256, 64 diffusion steps, confidence threshold 0.9, delimiter threshold 0.3, prefix+dual cache, minimum block length 1, minimum refinement budget 1, and no refinement offset. The final log file contains multiple runs; the reported numbers use the latest complete 200-prompt run, `0be5c8d2a37943949e33ae431bf63bd0`. The answer metric is exact presence of the accepted numeric answer in the generated text after normalization, matching the repository evaluator.

The average NFE ratio was 0.786, so PAG used about 21.4% fewer forward evaluations than AdaBlock. Median NFE delta was -18.000, and average runtime delta was -0.050 seconds. Runtime savings are smaller than NFE savings because predictor calls and Python control overhead are visible at this evaluation scale, but the model compute component still decreased.

Method	Accuracy	Correct	Avg. total NFE	Avg. blocks
PAG	89.500%	179/200	72.070	15.460
AdaBlock	89.500%	179/200	90.800	15.660

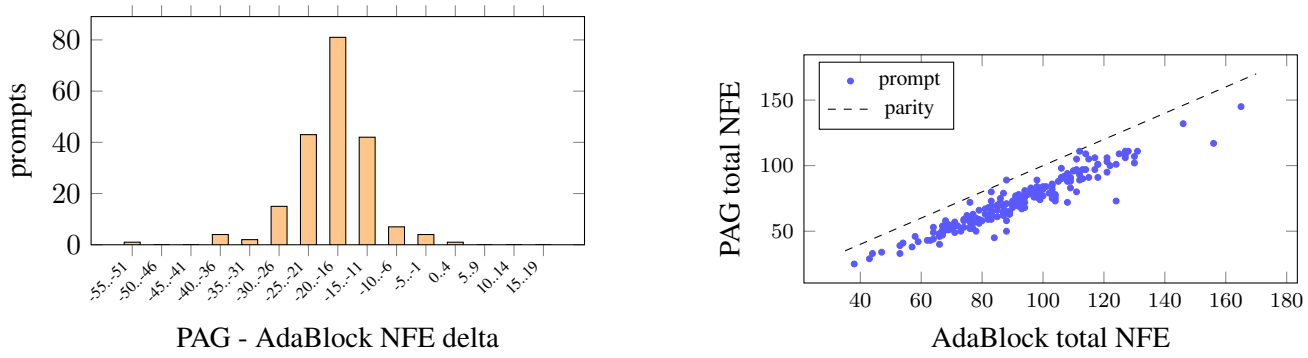
Table 2: Final 200-prompt GSM8K evaluation. PAG matches AdaBlock answer accuracy while using fewer forward evaluations.



(a) Average compute.

(b) Answer accuracy.

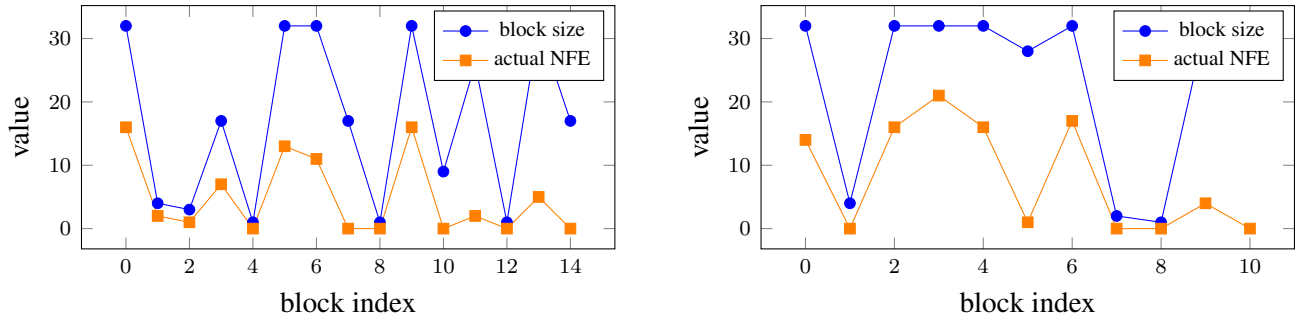
Figure 7: Aggregate final evaluation. PAG matches AdaBlock answer accuracy while using fewer forward evaluations.



(a) NFE savings distribution.

(b) Per-prompt compute parity.

Figure 8: Per-prompt final evaluation plots. PAG reduced NFE on 199 of 200 prompts, tied on 0, and used more NFE on 1.



(a) gsm8k_test_0370: -51 NFE delta.

(b) gsm8k_test_0374: +1 NFE delta.

Figure 9: Example PAG schedules from the final run. Most prompts save compute; the rare overhead cases occur when the learned schedule spends extra refinement on difficult blocks.

8 Discussion

What worked. The strongest empirical result is that learned tuple scheduling can preserve AdaBlock-level GSM8K answer accuracy while reducing compute. The result is not just an offline predictor metric: it holds inside the LLaDA decoding loop, with predicted tuples controlling actual unmasking. The trace analysis also produced a useful interpretability story: confidence and stabilization dynamics line up with intuitive easy and hard regions.

What did not fully work. Regression losses were misleading because the scheduler controls are discrete and multimodal. CPD also became less central than originally planned; it was valuable for understanding traces, but too sensitive for permanent label construction. The random forest baseline was interpretable but less natural for rollout because it depends on hand-designed rolling statistics and does not model sequence context as directly.

Why the final gains are plausible. AdaBlock reacts to current confidence. PAG inherits an AdaBlock-compatible seed and then uses learned history to choose later blocks and budgets. The final run shows that many prompts need systematically less refinement than AdaBlock allocates, while accuracy disagreements are rare: PAG loses 2 prompts that AdaBlock solves and gains 2 prompts that AdaBlock misses across the complete final run.

9 Limitations

Our answer metric is a normalized numeric substring check, so it can miss reasoning quality differences that do not change the final answer. We evaluate one model family, LLaDA-8B-Instruct, and one benchmark slice, GSM8K test[200:400]. HumanEval, MBPP, MATH, and Dream integration remain natural follow-up evaluations. Runtime results should also be interpreted carefully: Python-side scheduler overhead is not optimized, and a fused implementation could change the wall-clock tradeoff.

The predictor is trained from AdaBlock traces, so it learns to imitate and improve around AdaBlock's operating regime rather than discovering schedules from first principles. Finally, we do not yet have a full learned model for future confidence/digit/delimiter features under rollout; during real decoding those features are observed after each block, but offline cost-aware evaluation must partially teacher-force non-control fields.

10 Conclusion

PAG frames adaptive dLLM decoding as a next-control prediction problem. Across the project, we built trace collection, CPD analysis, block-native tuple datasets, multiple predictor families, LLaDA scheduler integration, visualization dashboards, and a 200-prompt final evaluation. The most important design lesson is that phase structure is real but should be attached to the scheduler's native control space. On the final held-out GSM8K run, the learned scheduler matched AdaBlock answer accuracy while reducing average NFE by about 21%, showing that lightweight history-conditioned control is a viable direction for diffusion LLM reasoning systems.

References

M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *ICLR (Oral)*, 2025.

S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, J. Han, H. Peng, and L. Kong. Scaling diffusion language models via adaptation from autoregressive models. *ICLR*, 2025.

R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012.

G. Lu, H. M. Chen, Y. Karashima, Z. Wang, D. Fujiki, and H. Fan. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. *ICLR*, 2026.

S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. *arXiv:2502.09992*, 2025.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *NeurIPS*, 2017.

C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *ICLR*, 2026.

J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong. Dream 7B: Diffusion large language models. *arXiv:2508.15487*, 2025.